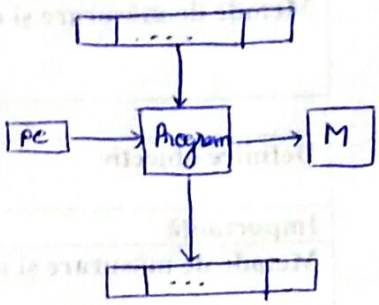
MODELUL RAM (RANDOM ACCESS MACHINE)

- Acest model este similar cu modelul mașinii Turing.
- Un astfel de model este utilizat pentru dezvoltarea de algoritmi și analiza complexității acestora, constituind punctul de plecare pentru modelele paralele
- Are → o bandă de intrare
  → o bandă de ieșire
  → program
  → program counter ⇒ folosit pentru execuția programului

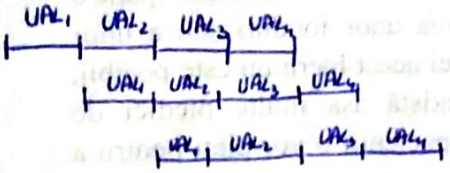


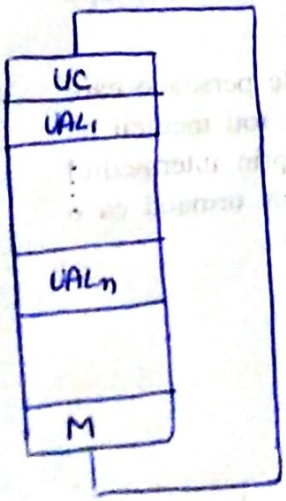
- RAM = $\{ S, \Gamma, \Sigma, f, s_0, Z \}$
  - S – mulțime finită a stărilor
  - Γ – alfabetul benzii
  - Σ – alfabetul benzii de intrare
  - f – funcție de tranziție: $f: S \times \Gamma \to S \times \Gamma$
  - s_0 – starea inițială ($s_0 \in S$)
  - Z – mulțimea stărilor finale ($Z \subseteq S$)

PIPELINE

Presupune divizarea unui task T în subtask-uri $T_1, T_2, \ldots, T_k$ și de a le atribui unor elemente de procesare.

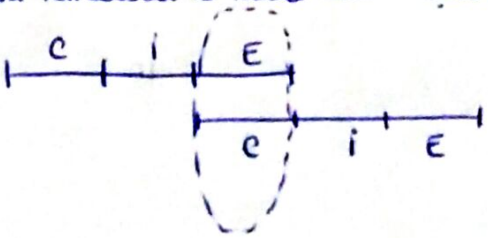
Paralelismul se poate realiza la nivel de → prelucrare aritmetică
→ citire, interpretare, execuție instrucțiuni





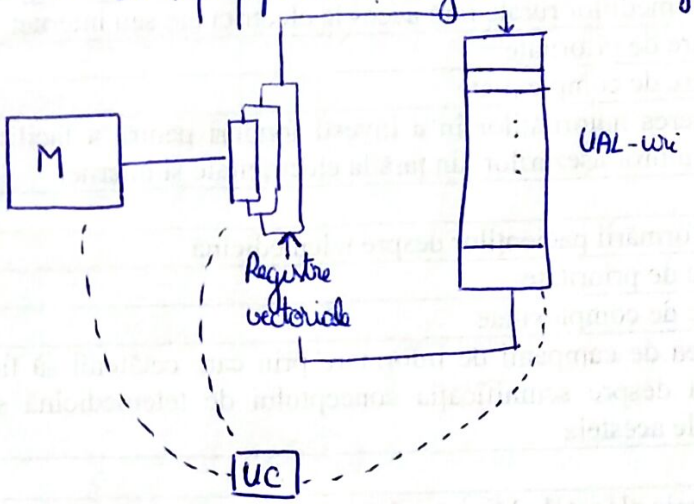
UC are mai multe faze: $UC_{f_1}, \ldots, UC_{f_n}$

Orice UC se ocupă de citirea, interpretarea și execuția instrucțiunilor mașină. După ce începe să execute, se poate apuca să citească o altă instrucțiune.



PROCESOARE DE VECTORI

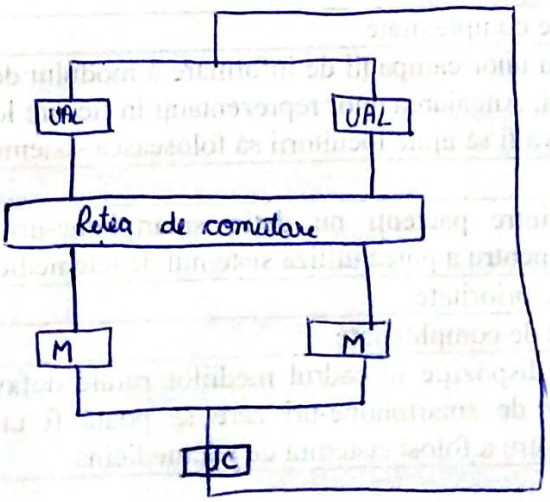
- există un set de instrucțiuni care tratează vectorul ca operand simplu
- există o UAL de tip pipeline și regiștri vectori (fiecare UAL se ocupă de un registru)



- poate fi asociată cu SIMD, dar spre deosebire de SIMD:
  - nu avem multiple unități de procesare
  - este o mașină secvențială, dar care mai multe UAL-uri

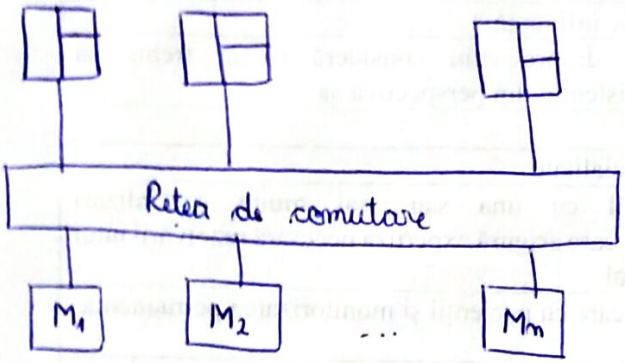
PROCESARE MASIVĂ DE DATE

- Se referă la calculatoare paralele sincrone, care constau din UAL multiple, supervizate de o singură UC și care efectuează aceeași operație la un moment dat.
- UAL efectuează aceeași operație asupra unor date diferite
- poate fi asociat cu un SIMD extins



MEMORIE PARTAJATĂ

- Fiecare procesor conține propria unitate de prelucrare, propria memorie și propria unitate de comandă
- Se comunică prin intermediul unei rețele de comutare cu module de memorie partajată.
- Se încadrează la NUMA

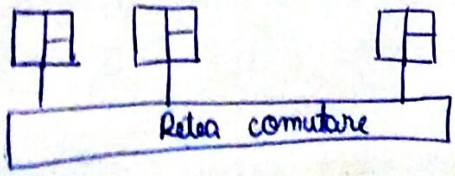


PRAM - STRUCTURĂ PARALELĂ CU ACCES ALEATOR

- extensie a RAM
- comunicație între procesoare și memorie fără introducere de întârziere
- memorie comună disponibilă tuturor programelor
- 4 modalități de acces la memorie → EREW
  → CREW
  → ERCW
  → CRCW

TRANSFER PRIN MESAJE

- Nu se mai partajează memoria
- Memoria este distribuită fiecărui procesor a.î. fiecare dispune de propriul program și propria memorie de date.
- Interconexiunea se realizează prin schimburi de mesaje prin intermediul unei rețele de comutare mesaje
- Exemplu: cluster (nu putem să vedem memoria celuilalt, dar folosim send și receive)



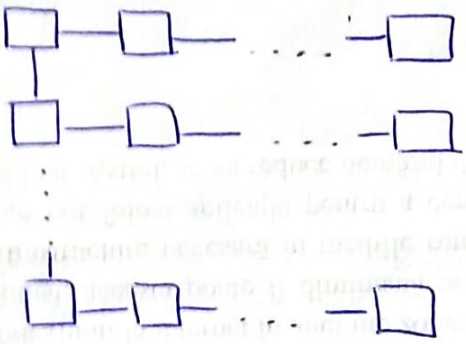
PROCESARE SISTOLICĂ

- constă în elemente de procesare identice aranjate într-o structură pipeline



PA = procesor aritmetic

Pipeline liniar

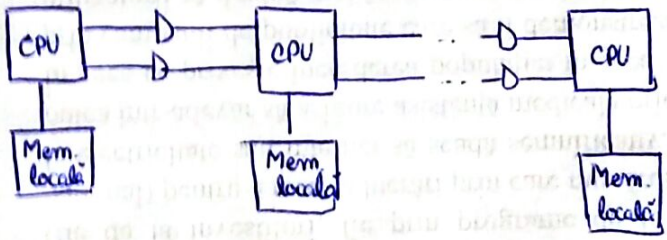


Pipeline matriceal

- Se caracterizează prin interconexiuni simple ce permite integrarea VLSI

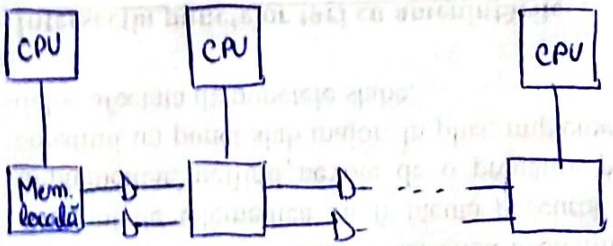
Metode de comunicare în sistemele sistolice:

(1)



După prelucrare, punem într-un buffer, iar următorul știe să ia de acolo.

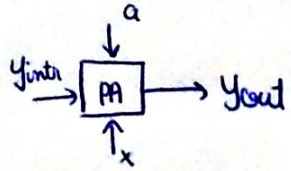
(2)

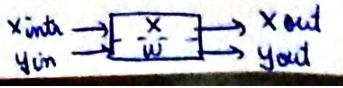


Prelucrez ceva și pun rezultatul în memorie. În acest moment există un acces direct de memorie care mută datele de ieșire în altă zonă de memorie ca fiind date de intrare ⇒ necesită implementarea unui DMA

Ex: $y = y_0 + Ax$

- ABORDARE 1: Secvențial ⇒ $O(n^2)$
- ABORDARE 2: Pipeline secvențial ⇒ Cost: $n \cdot PA$
  ⇒ $O(n)$
- ABORDARE 3: Pipeline matriceal ⇒ Cost: $n^2 \cdot PA$





DATA FLOW

- Specifică efectuarea unor operații imediat ce operanzii devin disponibili.
  ⇒ Se elimină necesitatea existenței contorului de program
- nu necesită adrese pentru memorarea variabilelor
- operația se efectuează numai când token-ul este prezent pe <u>ambele</u> intrări ale operandului
- datele se stochează în cadrul instrucțiunilor (se menține un token cu valoare binară în cadrul instrucțiunii pentru fiecare dată)
- disponibilitatea datelor este verificată de o unitate specializată, dedicată
- <u>Avantaje</u> → paralelism ridicat
  → viteză ridicată de execuție
  → ușor de implementat comunicația și sincronizarea
- <u>Dezavantaje</u> → overhead mare pentru control
  → manipulare dificilă a structurilor de date

- Token = { Data, Tag, Destinație, marker }
  Match = { Tag, Destinație }

